

CS634 Final Term Project Report

Name : Veekshitha Beladara Nanjegowda

Course : CS634 – Data Mining

Instructor : Dr. Yasser Abdulllah

Project : Diabetes Prediction Using Random Forest , LSTM, and SVM

Date: November 9, 2025

1. Introduction

This project aims to predict diabetes using binary classification Algorithms like Random Forest , Support Vector Machine , and Long Short-Term Memory. In healthcare binary classification is necessary where early detection determines treatment outcomes. Also used Pima Indians Diabetes datasets from the UCI Repository which contains 768 samples and 8 features. Preprocessing and feature normalization were also applied for effective training.

2. Dataset

Name: Pima Indians Diabetes Dataset (UCI Repository).

Link : <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

Records: 768 rows x 9 columns

Target variable: Outcome (0= No Diabetes, 1= Diabetes)

Preprocessing : Missing values replaced with median values for numerical columns also Data is standardized using StandardScaler.

Distribution of class: 65.1% No Diabetes , 34.9% Diabetes.

3. Algorithms Overview

Random Forest: Ensemble method builds multiple decision trees to reduce overfitting.

LSTM: A recurrent neural network that captures temporal dependencies which are used to test deep learning potential on structural data.

SVM : A kernel-based classifier maximizing class separation with strong generalization ability.

4. Implementation

Language : Python

Libraries: numpy, pandas, scikit-learn, tensorflow, keras, matplotlib, seaborn

Evaluation: 10-fold cross-validation with visualizations including heatmaps, ROC curves, and confusion matrices.

5. Evaluation Setup

Metrics: TP, TN, FP, FN, Accuracy, Precision, Recall, F1, FPR, FNR, Specificity, Balanced Accuracy, TSS, HSS, AUC, Brier Score, and BSS were computed manually and averaged across 10 folds.

6. Results

Average accuracy across 10 folds :

Random Forest : 75.65%

SVM: 76.69%

LSTM: 74.21%

Hence from the above observation, Random Forest had the best AUC(0.827), SVM has the best precision(0.7092), and LSTM achieved strong recall(0.60). All models displayed stable results.

=====	
LSTM - Average Results Across All Folds	
=====	
TP	: 16.1000
TN	: 40.9000
FP	: 9.1000
FN	: 10.7000
P	: 26.8000
N	: 50.0000
TPR	: 0.6000
TNR	: 0.8164
FPR	: 0.1836
FNR	: 0.4000
Precision	: 0.6381
Recall	: 0.6000
F1	: 0.6142
Accuracy	: 0.7421
Error Rate	: 0.2579
Balanced Acc	: 0.7082
TSS	: 0.4165
HSS	: 0.4198
AUC	: 0.8038
Brier Score	: 0.1716
BSS	: 0.2360

7. Discussion

SVM achieved the highest precision and accuracy, while Random Forest's AUC suggested superior generalization and LSTM slightly underperformed due to limited data size. Feature importance identified Glucose, BMI, and Age as dominant predictors. Challenges included managing missing values and maintaining consistency across each fold. Future improvements include hyperparameter tuning and dataset expansion.

8. Conclusion

Random Forest and SVM demonstrated reliable prediction capabilities on structured medical data. Deep learning shows promise with more data and tuning. The results validate the value of comparing multiple algorithms to ensure robust and explainable predictions in medical applications.

9. Reference

Dataset: <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

Scikit-learn Documentation : <https://scikit-learn.org>

TensorFlow Documentation : <https://www.tensorflow.org>

Research paper: "Early Diabetes Prediction Using ML and DL Approaches," IEEE, 2022.

GitHub Link: <https://github.com/veeksh07/Beladara-nanjegowd-final-project/tree/main>

Screenshot:

3. Data Preprocessing

```
columns_to_fix = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in columns_to_fix:
    data[col] = data[col].replace(0, np.nan)
    data[col] = data[col].fillna(data[col].median())

print("Missing values handled successfully!")
print("\nChecking for any remaining missing values:")
print(data.isnull().sum())
```

Python

Missing values handled successfully!

Checking for any remaining missing values:
Pregnancies 0
Glucose 0
BloodPressure 0
SkinThickness 0
Insulin 0
BMI 0
DiabetesPedigreeFunction 0
Age 0
Outcome 0
dtype: int64

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
Column Non-Null Count Dtype
--- ---
0 Pregnancies 768 non-null int64
1 Glucose 768 non-null int64
2 BloodPressure 768 non-null int64
3 SkinThickness 768 non-null int64
...
25% 27.300000 0.243750 24.000000 0.000000
50% 32.000000 0.372500 29.000000 0.000000
75% 36.600000 0.626250 41.000000 1.000000
max 67.100000 2.420000 81.000000 1.000000
[Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings.](#)

3. Data Preprocessing

```
columns_to_fix = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in columns_to_fix:
    data[col] = data[col].replace(0, np.nan)
    data[col] = data[col].fillna(data[col].median())

print("Missing values handled successfully!")
print("\nChecking for any remaining missing values:")
print(data.isnull().sum())
```

Python

Missing values handled successfully!

Checking for any remaining missing values:
Pregnancies 0
Glucose 0
BloodPressure 0

2. Load and Explore Data

```
data = pd.read_csv("diabetes.csv")

print("First 5 rows of data:")
print(data.head())

print("\nDataset Information:")
print(data.info())

# Statistical summary
print("\nStatistical Summary:")
print(data.describe())
```

[30]

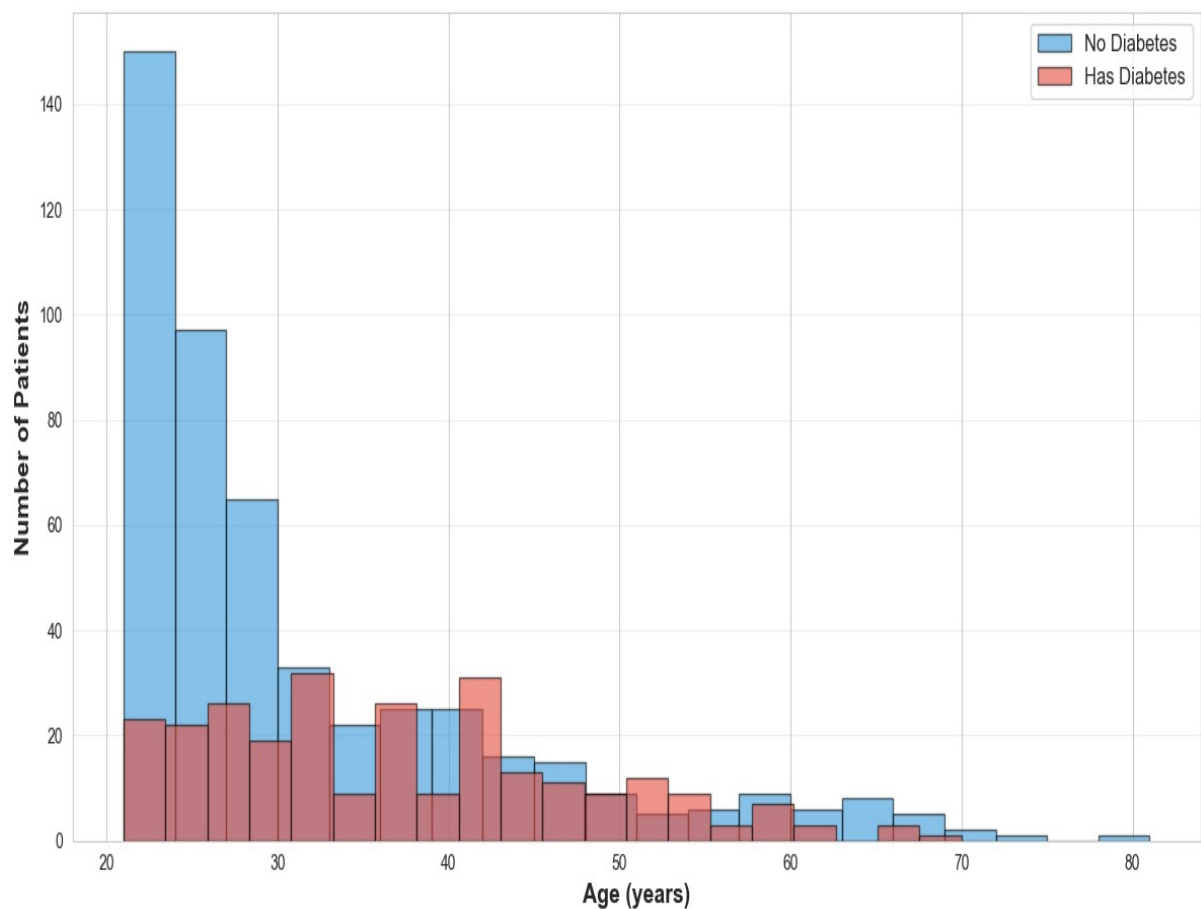
Python

```
-- First 5 rows of data:
Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  BMI  \
0           6       140             72           35         0   33.6
1           1        85             66           29         0   26.6
2           8       183             64           0         0   23.3
3           1        89             66           23         96   28.1
4           0       137             40           35       168   43.1

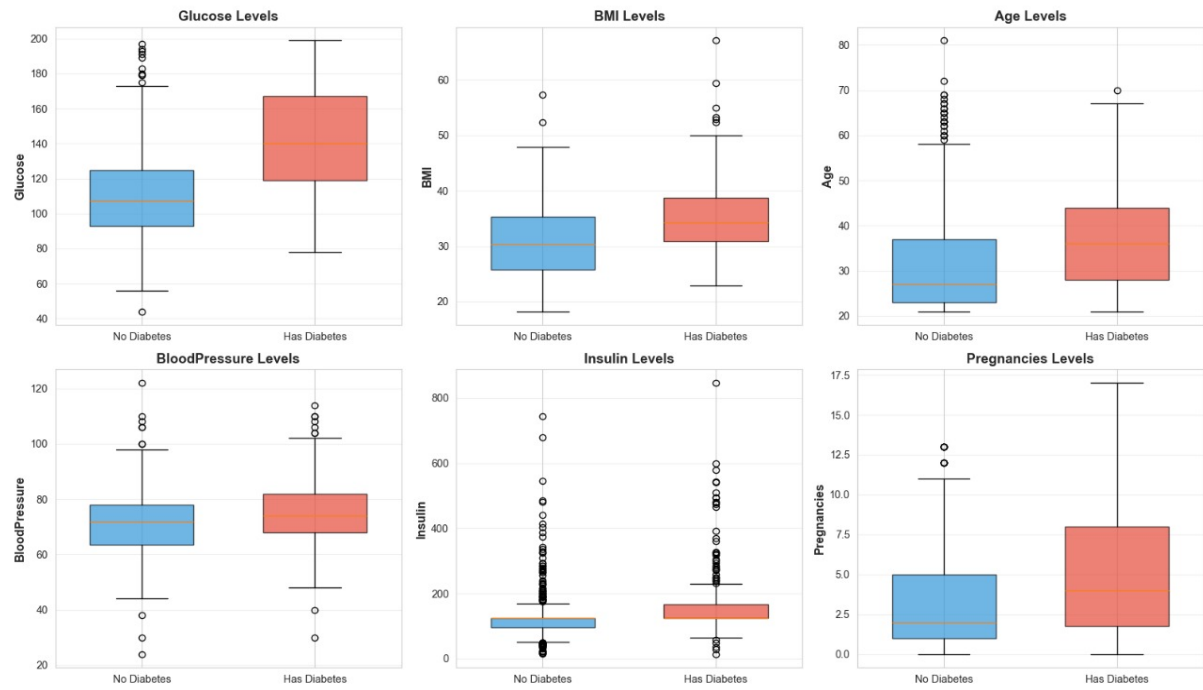
DiabetesPedigreeFunction  Age  Outcome
0              0.627      50         1
1              0.351      31         0
2              0.672      32         1
3              0.167      21         0
4              2.288      33         1

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column              Non-Null Count  Dtype
# 0  Pregnancies          768 non-null    int64
# 1  Glucose              768 non-null    int64
# 2  BloodPressure        768 non-null    int64
# 3  SkinThickness        768 non-null    int64
# 4  Insulin              768 non-null    int64
# 5  BMI                  768 non-null    float64
# 6  DiabetesPedigreeFunction  768 non-null    float64
# 7  Age                  768 non-null    int64
# 8  Outcome              768 non-null    int64
```

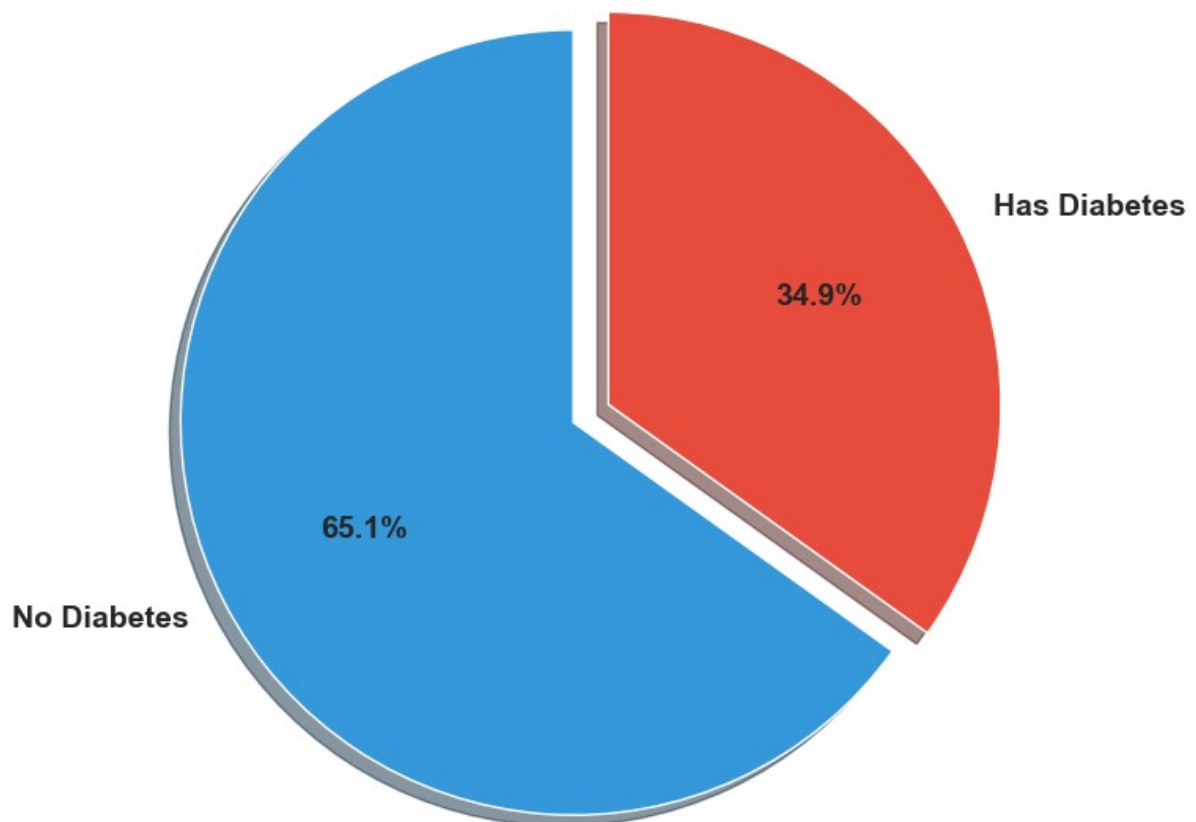
Age Distribution by Diabetes Status



Key Medical Features by Diabetes Status



Diabetes Distribution in Dataset



```
lstm_results.append(lstm_metrics)
print(f'LSTM Accuracy: {lstm_metrics['Accuracy']:.4f}')
print()

print(f'\n{"="*60}')
print("All folds completed successfully!")
print(f'{"="*60}')
```

[38]

Python

... Starting 10-Fold Cross-Validation...

=====

Processing Fold 1/10

=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.6883

[2/3] Training SVM...

SVM Accuracy: 0.7403

[3/3] Training LSTM...

LSTM Accuracy: 0.6883

=====

Processing Fold 2/10

=====

Train size: 691, Test size: 77

[1/3] Training Random Forest...

Random Forest Accuracy: 0.7922

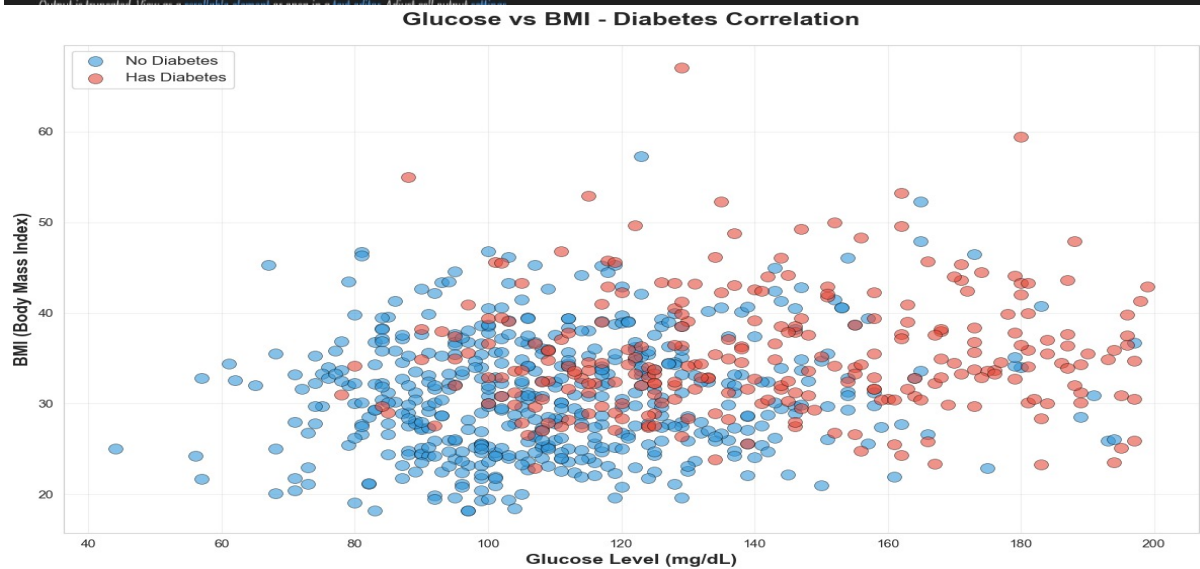
[2/3] Training SVM...

...

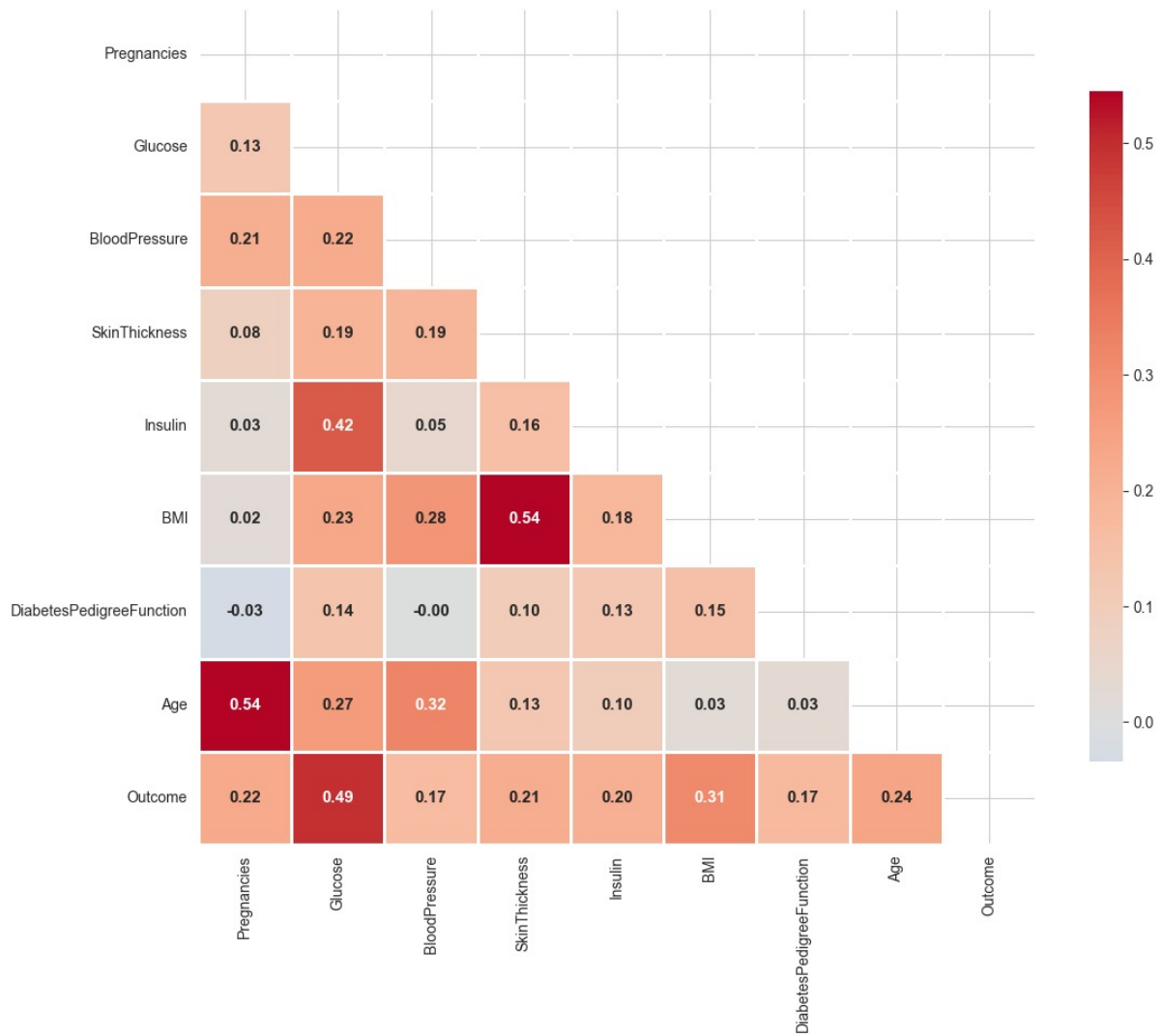
=====

All folds completed successfully!

=====



Feature Correlation Heatmap



LSTM - Per-Fold Results

TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Precision	Recall	F1	Accuracy	Error_Rate	Balanced_Acc	TSS	HSS	AUC	Brier_Score	BSS	Fold
18	35	15	9	27	58	0.6667	0.7000	0.3000	0.3333	0.5455	0.6667	0.6000	0.6883	0.3117	0.6813	0.3667	0.2488	0.7533	0.1963	0.1379	1
22	37	12	6	28	49	0.7857	0.7551	0.2449	0.2143	0.6471	0.7857	0.7097	0.7662	0.2338	0.7704	0.5408	0.5171	0.8703	0.1493	0.3550	2
12	44	8	13	25	52	0.4800	0.8662	0.1538	0.5200	0.6000	0.4800	0.5333	0.7273	0.2727	0.6631	0.3262	0.3440	0.7300	0.2055	0.0629	3
15	46	9	7	22	55	0.6818	0.3264	0.1636	0.3182	0.6250	0.6818	0.6522	0.7022	0.2978	0.7591	0.5182	0.5044	0.9455	0.1438	0.2956	4
18	44	5	10	28	49	0.6429	0.8980	0.1020	0.3571	0.7826	0.6429	0.7059	0.8052	0.1948	0.7704	0.5408	0.5623	0.8491	0.1539	0.3350	5
18	34	10	15	33	44	0.5455	0.7727	0.2273	0.4545	0.6429	0.5455	0.5902	0.6753	0.3247	0.6591	0.3182	0.3243	0.7514	0.2095	0.1447	6
19	44	6	8	27	50	0.7037	0.8800	0.1200	0.2963	0.7600	0.7037	0.7388	0.8182	0.1818	0.7919	0.5837	0.5938	0.8896	0.1328	0.4169	7
9	49	8	11	20	57	0.4500	0.8596	0.1404	0.5500	0.5294	0.4500	0.4865	0.7532	0.2468	0.6548	0.3096	0.3255	0.7640	0.1660	0.1367	8
14	36	9	17	31	45	0.4516	0.8000	0.2000	0.5484	0.6087	0.4516	0.5185	0.6579	0.3421	0.6258	0.2516	0.2621	0.7756	0.1925	0.2030	9
16	40	9	11	27	49	0.5926	0.8163	0.1837	0.4074	0.6400	0.5926	0.6154	0.7368	0.2632	0.7045	0.4089	0.4158	0.8095	0.1666	0.2725	10

LSTM - Average Results Across All Folds

TP	: 16.1000
TN	: 46.9000
FP	: 9.1000
FN	: 10.7000
P	: 26.8000
N	: 50.0000
HSS	: 0.4198
AUC	: 0.8038
Brier_Score	: 0.1716
BSS	: 0.2360

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#).

10. Compare All Models - Summary Table

===== SVM - Per-Fold Results =====

TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Precision	Recall	F1	Accuracy	Error_Rate	Balanced_Acc	TSS	HSS	AUC	Brier_Score	BSS	Fold
16	41	9	11	27	50	0.5926	0.8200	0.1800	0.4074	0.6400	0.5926	0.6154	0.7403	0.2597	0.7063	0.4126	0.4197	0.7800	0.1789	0.2143	1
17	45	4	11	28	49	0.6071	0.9184	0.0816	0.3929	0.8095	0.6071	0.6939	0.8052	0.1948	0.7628	0.5255	0.5553	0.8258	0.1497	0.3533	2
14	43	9	11	25	52	0.5600	0.8269	0.1731	0.4400	0.6087	0.5600	0.5833	0.7403	0.2597	0.6935	0.3869	0.3951	0.7531	0.1929	0.1202	3
14	51	4	8	22	55	0.6364	0.9273	0.0727	0.3636	0.7778	0.6364	0.7000	0.8442	0.1558	0.7818	0.5636	0.5962	0.9017	0.1195	0.4143	4
18	43	6	10	28	49	0.6429	0.8776	0.1224	0.3571	0.7500	0.6429	0.6923	0.7922	0.2078	0.7602	0.5204	0.5368	0.8812	0.1410	0.3907	5
15	40	4	18	33	44	0.4545	0.9091	0.0909	0.5455	0.7895	0.4545	0.5769	0.7143	0.2857	0.6818	0.3636	0.3840	0.7775	0.1919	0.2162	6
17	44	6	10	27	50	0.6296	0.8800	0.1200	0.3704	0.7391	0.6296	0.6800	0.7922	0.2078	0.7548	0.5096	0.5276	0.8785	0.1483	0.3487	7
6	51	6	14	20	57	0.3000	0.8947	0.1053	0.7000	0.5000	0.3000	0.3750	0.7403	0.2597	0.5974	0.1947	0.2238	0.7868	0.1676	0.1281	8
16	39	6	15	31	45	0.5161	0.8667	0.1333	0.4839	0.7273	0.5161	0.6038	0.7237	0.2763	0.6914	0.3828	0.4009	0.7885	0.1965	0.1864	9
15	44	5	12	27	49	0.5556	0.8980	0.1020	0.4444	0.7500	0.5556	0.6383	0.7763	0.2237	0.7268	0.4535	0.4815	0.8632	0.1446	0.3685	10

===== SVM - Average Results Across All Folds =====

TP : 14.8000
TN : 44.1000
FP : 5.9000
FN : 12.0000
P : 26.8000
N : 50.0000
...
HSS : 0.4521
AUC : 0.8236
Brier_Score : 0.1631
BSS : 0.2741

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings...

===== RANDOM FOREST - Per-Fold Results =====

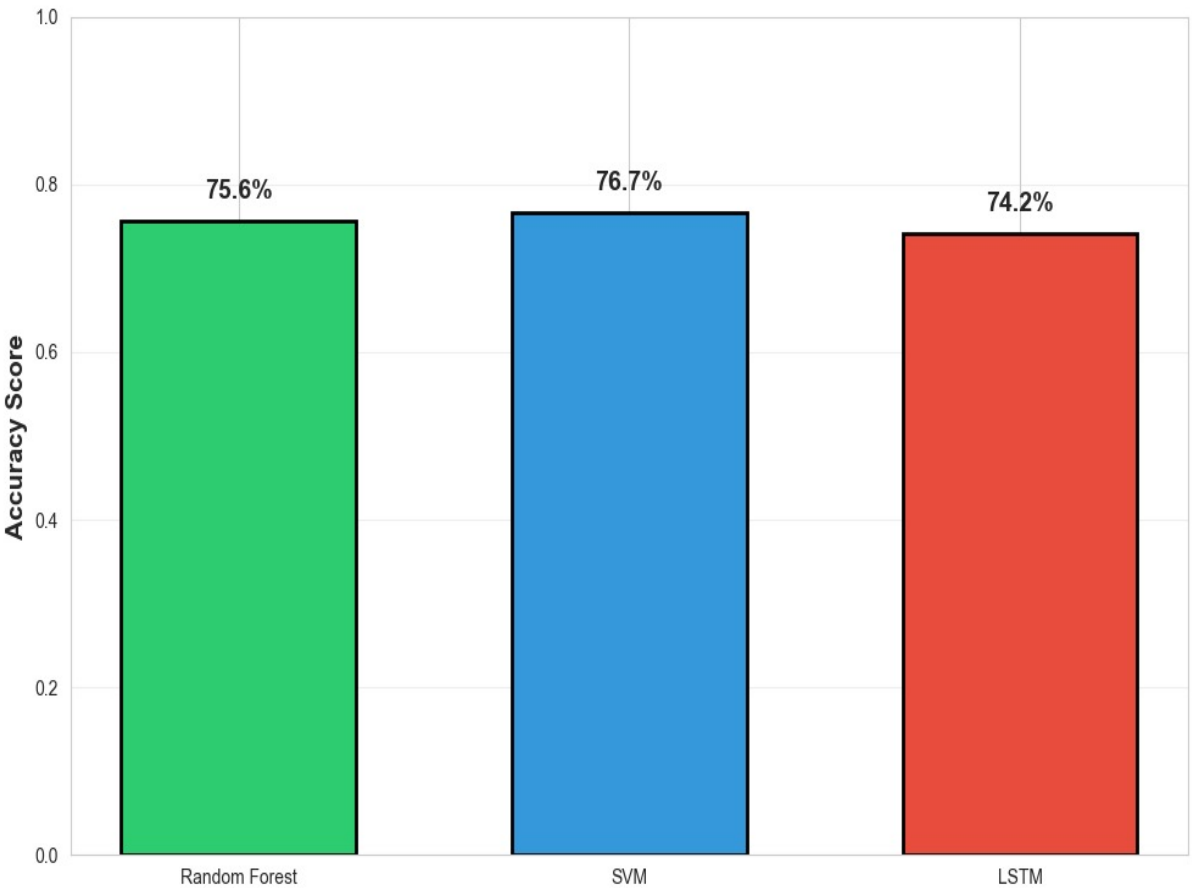
TP	TN	FP	FN	P	N	TPR	TNR	FPR	FNR	Precision	Recall	F1	Accuracy	Error_Rate	Balanced_Acc	TSS	HSS	AUC	Brier_Score	BSS	Fold
16	37	13	11	27	50	0.5926	0.7400	0.2600	0.4074	0.5517	0.5926	0.5714	0.6883	0.3117	0.6663	0.3326	0.3270	0.7941	0.1796	0.2112	1
20	41	8	8	28	49	0.7143	0.8367	0.1633	0.2857	0.7143	0.7143	0.7143	0.7922	0.2078	0.7755	0.5510	0.5510	0.8794	0.1401	0.3945	2
15	42	10	10	25	52	0.6000	0.8077	0.1923	0.4000	0.6000	0.6000	0.6000	0.7403	0.2597	0.7038	0.4077	0.4077	0.7235	0.1989	0.0929	3
15	48	7	7	22	55	0.6818	0.8727	0.1273	0.3182	0.6818	0.6818	0.6818	0.8182	0.1818	0.7773	0.5545	0.5545	0.9074	0.1166	0.4286	4
19	44	5	9	28	49	0.6786	0.8900	0.1020	0.3214	0.7917	0.6786	0.7308	0.8182	0.1818	0.7883	0.5765	0.5947	0.8848	0.1370	0.4078	5
14	39	5	19	33	44	0.4242	0.8864	0.1136	0.5758	0.7368	0.4242	0.5385	0.6883	0.3117	0.6553	0.3106	0.3280	0.7493	0.2100	0.1425	6
20	42	8	7	27	50	0.7407	0.8400	0.1600	0.2593	0.7143	0.7407	0.7273	0.8052	0.1948	0.7904	0.5807	0.5758	0.8922	0.1245	0.4532	7
8	48	9	12	20	57	0.4000	0.8421	0.1579	0.6000	0.4706	0.4000	0.4324	0.7273	0.2727	0.6211	0.2421	0.2545	0.7702	0.1702	0.1150	8
15	35	10	16	31	45	0.4839	0.7778	0.2222	0.5161	0.6000	0.4839	0.5357	0.6579	0.3421	0.6308	0.2616	0.2698	0.7849	0.1908	0.2098	9
17	46	3	10	27	49	0.6296	0.9388	0.0612	0.3704	0.8500	0.6296	0.7234	0.8289	0.1711	0.7842	0.5684	0.6035	0.8840	0.1338	0.4157	10

===== RANDOM FOREST - Average Results Across All Folds =====

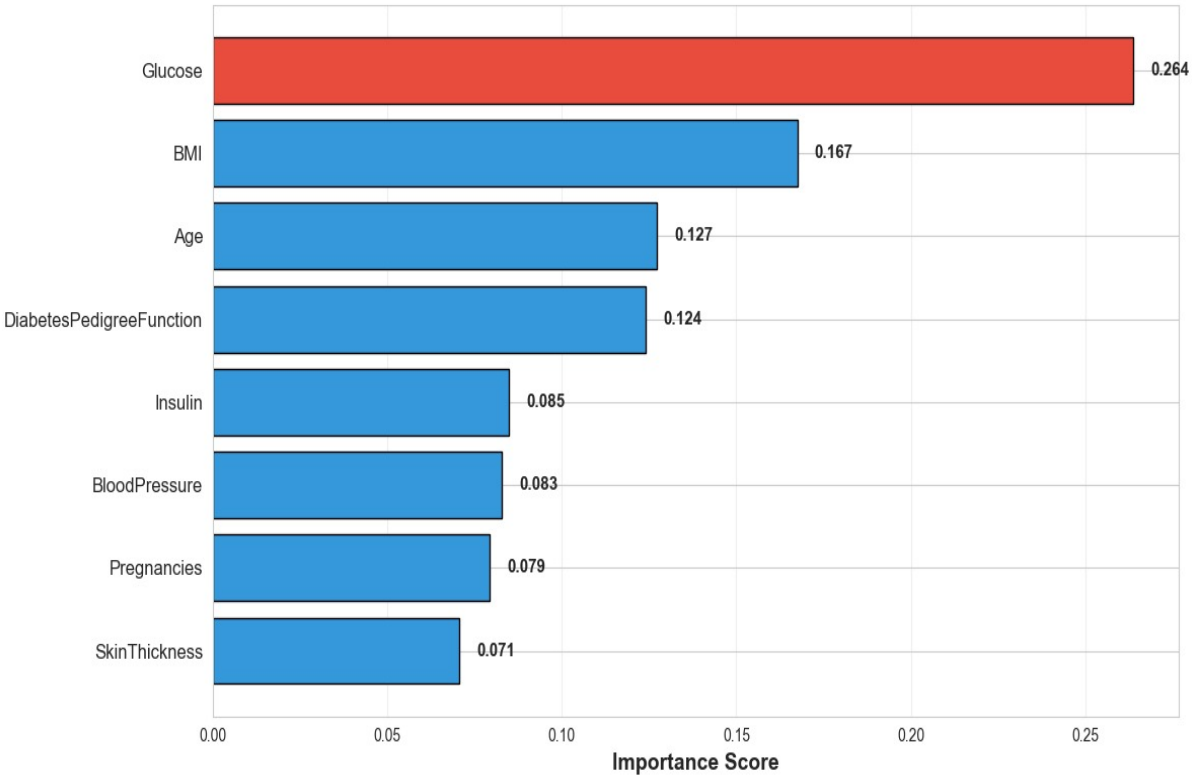
TP : 15.9000
TN : 42.2000
FP : 7.8000
FN : 10.9000
P : 26.8000
N : 50.0000
...
HSS : 0.4467
AUC : 0.8270
Brier_Score : 0.1602
BSS : 0.2871

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings...

Model Performance Comparison



Feature Importance for Diabetes Prediction



```

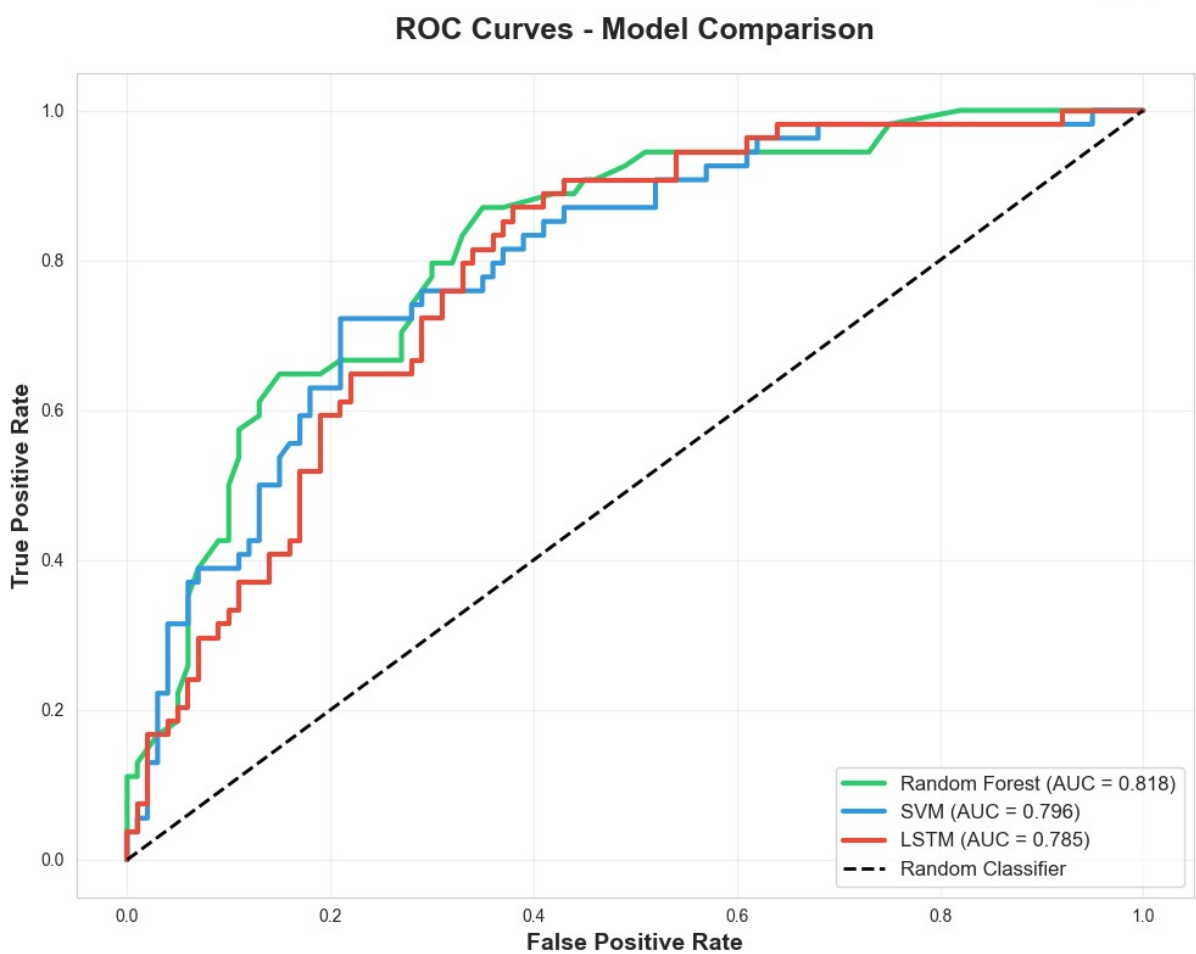
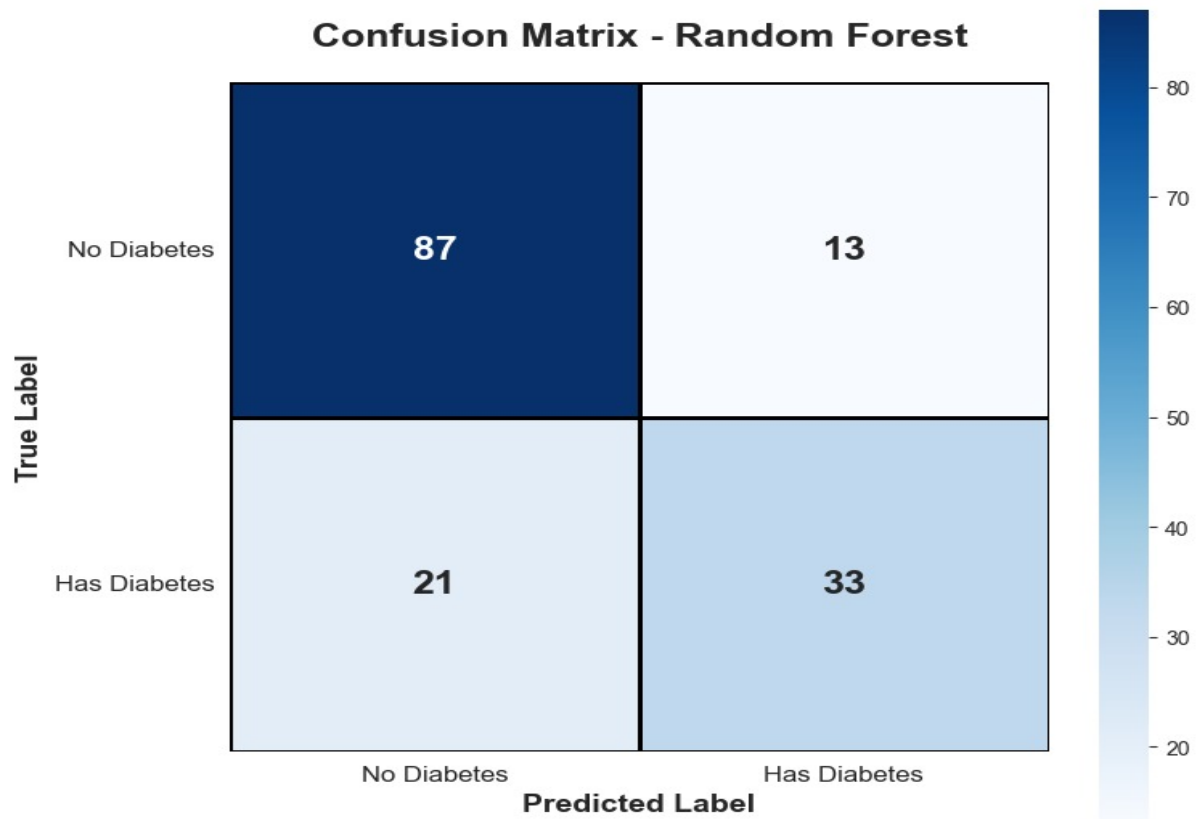
=====
MODEL COMPARISON - Average Performance Across 10 Folds
=====

```

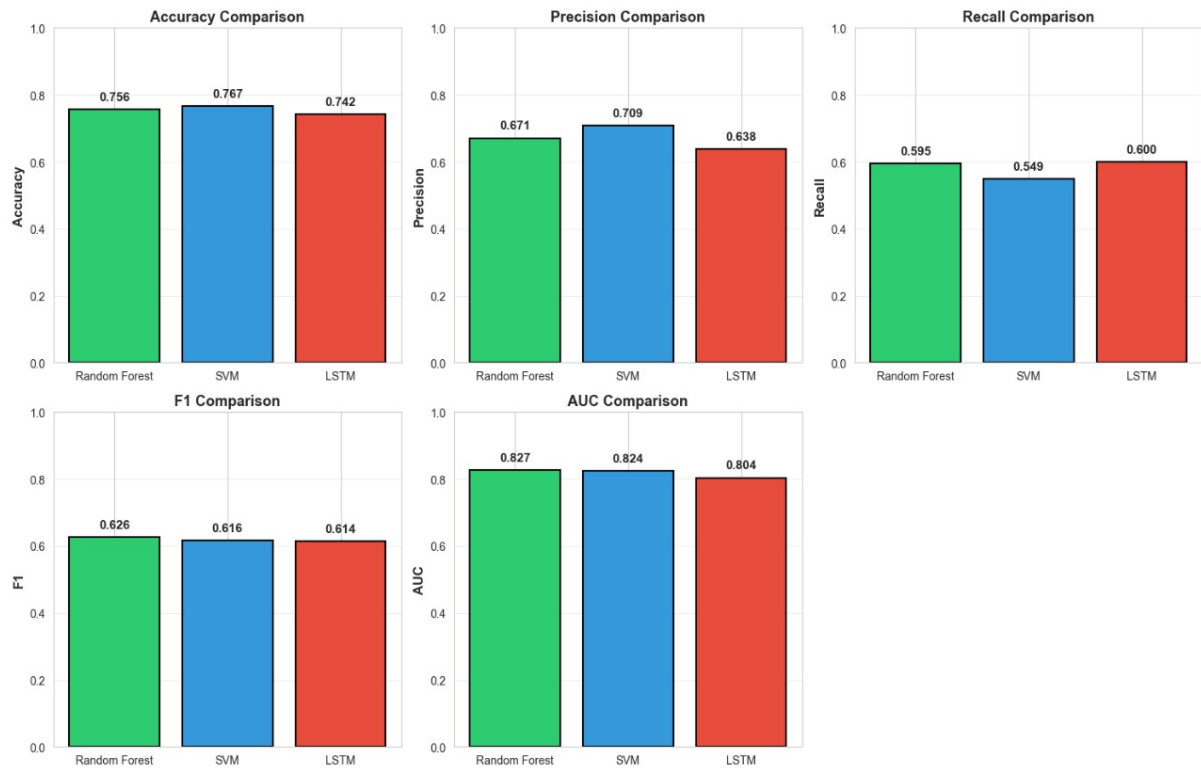
	Random_Forest	SVM	LSTM
TP	15.9000	14.8000	16.1000
TN	42.2000	44.1000	40.9000
FP	7.8000	5.9000	9.1000
FN	10.9000	12.0000	10.7000
P	26.8000	26.8000	26.8000
N	50.0000	50.0000	50.0000
TPR	0.5946	0.5495	0.6000
TNR	0.8440	0.8819	0.8164
FPR	0.1560	0.1181	0.1836
FNR	0.4054	0.4505	0.4000
Precision	0.6711	0.7092	0.6381
Recall	0.5946	0.5495	0.6000
F1	0.6256	0.6159	0.6142
Accuracy	0.7565	0.7669	0.7421
Error_Rate	0.2435	0.2331	0.2579
Balanced_Acc	0.7193	0.7157	0.7082
TSS	0.4386	0.4313	0.4165
HSS	0.4467	0.4521	0.4198
AUC	0.8270	0.8236	0.8038
Brier_Score	0.1602	0.1631	0.1716
...			
HSS	: SVM	(0.4521)	
AUC	: Random_Forest	(0.8270)	
Brier_Score	: LSTM	(0.1716)	
BSS	: Random_Forest	(0.2871)	

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#).

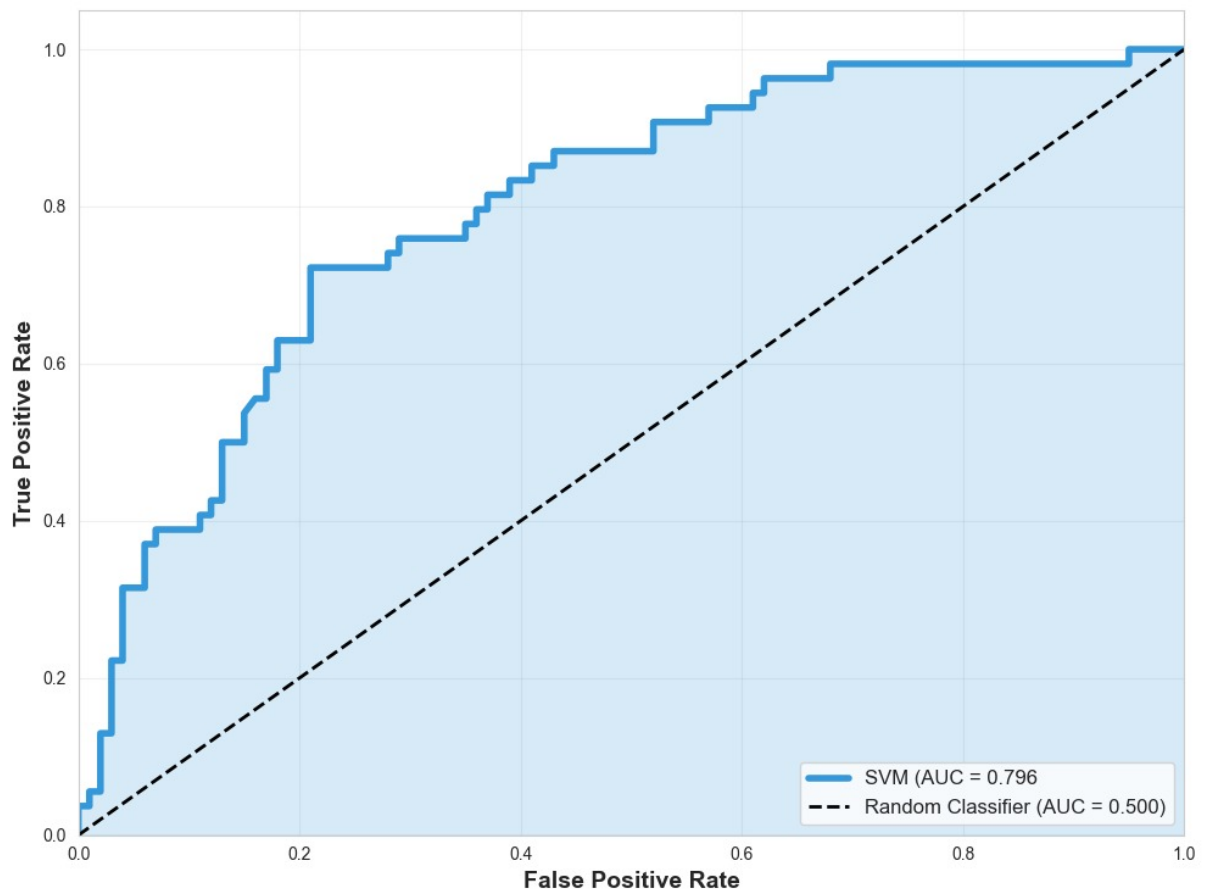




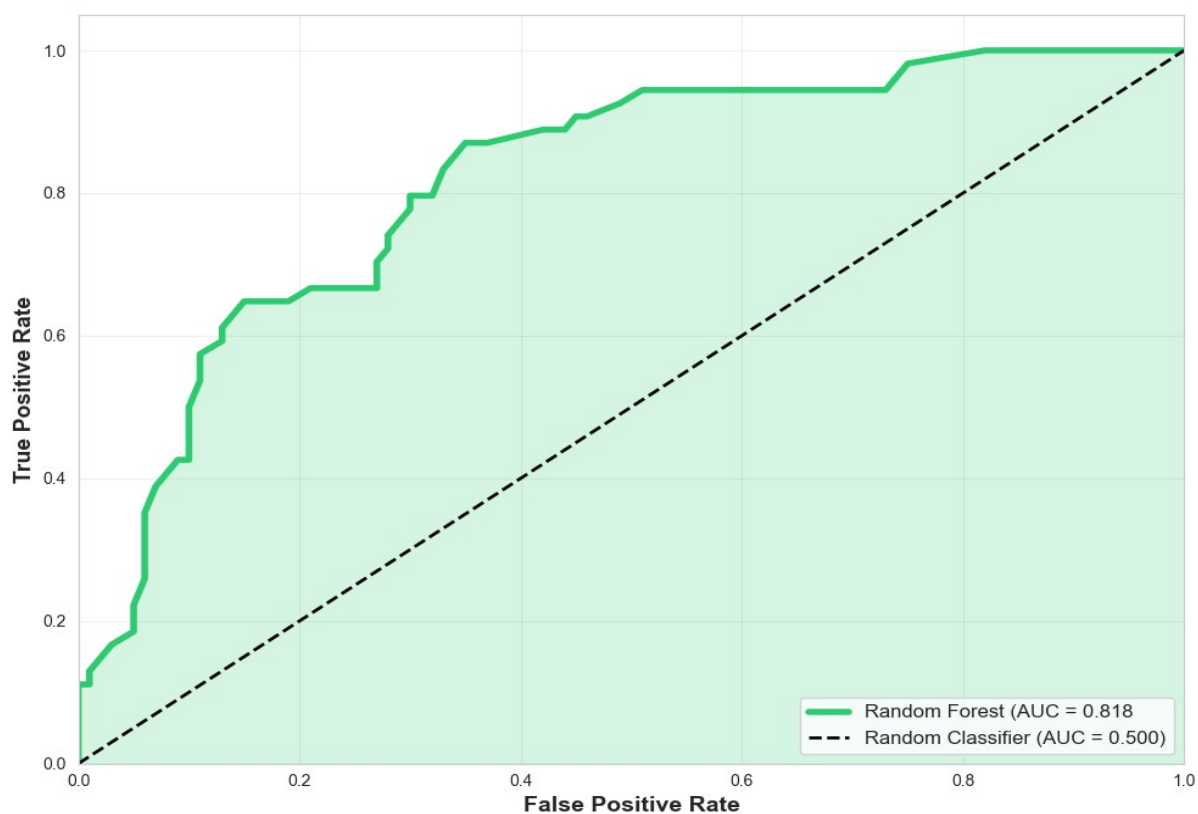
Key Performance Metrics - Model Comparison



ROC Curve - Support Vector Machine



ROC Curve - Random Forest



ROC Curve - LSTM Neural Network

